

Sorting

4/23/25

A common problem: Sorting

- Have collection of objects (numbers, strings, dates, ...) and want to put them into a defined order
- How do we specify the order?
- How do we do this efficiently?
 - assume objects are in an array
 - assume you can compare any pair of objects

Generalizing comparisons

- Comparable<T> interface

int compareTo(T other)

negative: this < other

positive: this > other

zero: this == other

- Comparator<T> interface

int compare(T o1, T o2)

as above, with o1 taking role of “this”

Selection sort

(code adapted from Wikipedia)

- Grow sorted part of array by finding smallest value in the remaining

```
for(int i=0; i < A.length-1; i++) {  
    int minIndex = i;           //index of min  
    for(int j=i+1; j < A.length; j++) { //find the min  
        if(A[j] < A[minIndex])  
            minIndex = j;  
    }  
    swap A[i] and A[minIndex];  
}
```

Selection sort

(code adapted from Wikipedia)

- Grow sorted part of array by finding smallest value in the remaining

```
for(int i=0; i < A.length-1; i++) {  
    int minIndex = i;           //index of min  
    for(int j=i+1; j < A.length; j++) { //find the min  
        if(A[j] < A[minIndex])  
            minIndex = j;  
    }  
    swap A[i] and A[minIndex];  
}
```

What most accurately characterizes the running time of selection sort?

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(n^3)$
- E. None of the above

Selection sort

(code adapted from Wikipedia)

- Grow sorted part of array by finding smallest value in the remaining

```
for(int i=0; i < A.length-1; i++) {  
    int minIndex = i;           //index of min  
    for(int j=i+1; j < A.length; j++) { //find the min  
        if(A[j] < A[minIndex])  
            minIndex = j;  
    }  
    swap A[i] and A[minIndex];  
}
```

What most accurately characterizes the running time of selection sort?

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(n^3)$
- E. None of the above

Insertion sort

(code adapted from Wikipedia)

- Keep sorted portion of the array and grow it by inserting new values into the proper place

```
for(int i=0; i < A.length; i++) {    //add value A[i]
    int j = i;                        //iterator to seek possible places
    while((j > 0) && (A[j-1] > A[j])) {
        swap A[j] and A[j-1];
        j--;
    }
}
```

Insertion sort

(code adapted from Wikipedia)

- Keep sorted portion of the array and grow it by inserting new values into the proper place

```
for(int i=0; i < A.length; i++) {    //add value A[i]
    int j = i;                        //iterator to seek possible places
    while((j > 0) && (A[j-1] > A[j])) {
        swap A[j] and A[j-1];
        j--;
    }
}
```

What most accurately characterizes the running time of insertion sort?

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(n^3)$
- E. None of the above

Insertion sort

(code adapted from Wikipedia)

- Keep sorted portion of the array and grow it by inserting new values into the proper place

```
for(int i=0; i < A.length; i++) {    //add value A[i]
    int j = i;                        //iterator to seek possible places
    while((j > 0) && (A[j-1] > A[j])) {
        swap A[j] and A[j-1];
        j--;
    }
}
```

What most accurately characterizes the running time of insertion sort?

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(n^3)$
- E. None of the above

Bubble sort

(code adapted from Wikipedia)

- Repeatedly find out-of-order pairs and swap them

```
boolean swapped = true;
while(swapped) {
    swapped = false;
    for(int i=0; i < A.length-1; i++)
        if(A[i] > A[i+1]) {
            swap A[i] and A[i+1];
            swapped = true;
        }
}
```

Bubble sort

(code adapted from Wikipedia)

- Repeatedly find out-of-order pairs and swap them

```
boolean swapped = true;
while(swapped) {
    swapped = false;
    for(int i=0; i < A.length-1; i++)
        if(A[i] > A[i+1]) {
            swap A[i] and A[i+1];
            swapped = true;
        }
}
```

What most accurately characterizes the running time of bubble sort?

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(n^3)$
- E. None of the above

Bubble sort

(code adapted from Wikipedia)

- Repeatedly find out-of-order pairs and swap them

```
boolean swapped = true;
while(swapped) {
    swapped = false;
    for(int i=0; i < A.length-1; i++)
        if(A[i] > A[i+1]) {
            swap A[i] and A[i+1];
            swapped = true;
        }
}
```

What most accurately characterizes the running time of bubble sort?

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(n^3)$
- E. None of the above

Summary: Basic algorithms

- Selection sort
 - Grow sorted part of array by finding smallest value in the remaining
- Insertion sort
 - Grow sorted part of the array by inserting new values into the proper place
- Bubble sort
 - Repeatedly find out-of-order pairs and swap them

Which of the sorting algorithms could cause the execution shown below?

5	1	4	2	3
1	5	4	2	3
1	2	4	5	3
1	2	3	5	4

- A. Selection sort
- B. Insertion sort
- C. Bubble sort
- D. None of these
- E. More than one of these

Which of the sorting algorithms could cause the execution shown below?

5	1	4	2	3
1	5	4	2	3
1	2	4	5	3
1	2	3	5	4

A. Selection sort

B. Insertion sort

C. Bubble sort

D. None of these

E. More than one of these

Which of the sorting algorithms could cause the execution shown below?

5	1	4	2	3
1	5	4	2	3
1	4	5	2	3
1	4	2	5	3

- A. Selection sort
- B. Insertion sort
- C. Bubble sort
- D. None of these
- E. More than one of these

Which of the sorting algorithms could cause the execution shown below?

5	1	4	2	3
1	5	4	2	3
1	4	5	2	3
1	4	2	5	3

A. Selection sort

B. Insertion sort

C. Bubble sort

D. None of these

E. More than one of these (insertion & bubble)

Summary of basic sorting algorithms

- Assume data is in an array and have comparison operator (comparator)
- Selection sort
 - Grow sorted part of array by finding smallest value in the remaining
- Insertion sort
 - Grow sorted part of the array by inserting new values into the proper place
- Bubble sort
 - Repeatedly find out-of-order pairs and swap them

Can we do better than $O(n^2)$ time?

Merge sort

- Split array into two equal-sized pieces, recursively sort each half, and merge them back

```
Mergesort(A[1..n]) {  
    if(n > 1) {  
        m = (n+1)/2;  
        copy 1st m values of A into array L and rest into array R;  
        Mergesort(L);  
        Mergesort(R);  
        Merge(L, R, A);  
    }  
}
```

```

Merge(A[1..n1], B[1..n2], R[1..(n1+n2)]) {
    //merge sorted arrays A and B into R
    int i = 1, j = 1;           //i is position in A, j is position in B
    for(int k = 1; k <= (n1+n2); k++) { //k is position in R
        if(i ≤ n1 and (j > n2 or A[i] ≤ B[j])) {
            R[k] = A[i];
            i++;
        } else {
            R[k] = B[j];
            j++;
        }
    }
}

```

```

Merge(A[1..n1], B[1..n2], R[1..(n1+n2)]) {
    //merge sorted arrays A and B into R
    int i = 1, j = 1;           //i is position in A, j is position in B
    for(int k = 1; k <= (n1+n2); k++) { //k is position in R
        if(i ≤ n1 and (j > n2 or A[i] ≤ B[j])) {
            R[k] = A[i];
            i++;
        } else {
            R[k] = B[j];
            j++;
        }
    }
}

```

What most accurately characterizes the running time of Merge? ($n = n_1 + n_2$)

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. None of the above

```

Merge(A[1..n1], B[1..n2], R[1..(n1+n2)]) {
    //merge sorted arrays A and B into R
    int i = 1, j = 1;           //i is position in A, j is position in B
    for(int k = 1; k <= (n1+n2); k++) { //k is position in R
        if(i ≤ n1 and (j > n2 or A[i] ≤ B[j])) {
            R[k] = A[i];
            i++;
        } else {
            R[k] = B[j];
            j++;
        }
    }
}

```

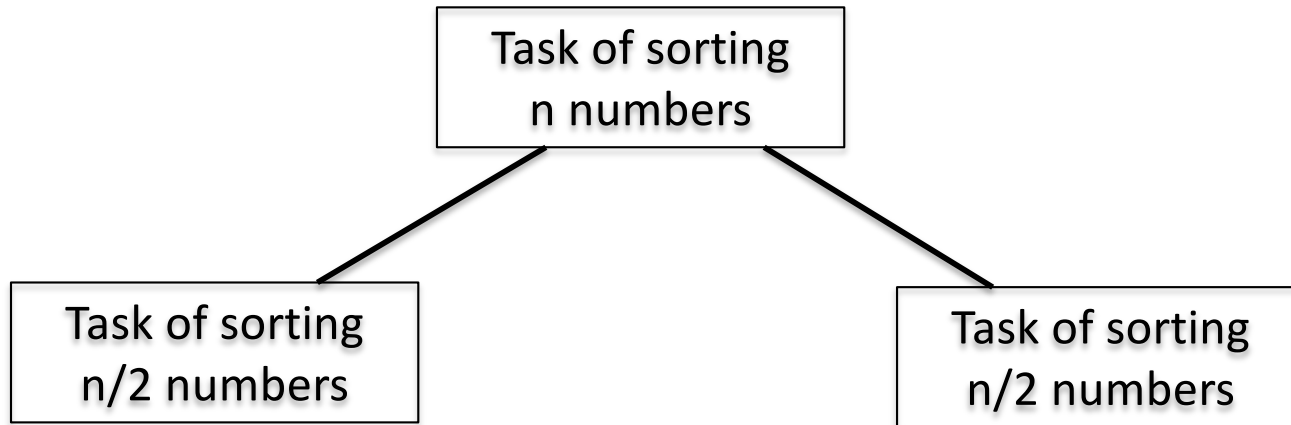
What most accurately characterizes the running time of Merge? ($n = n_1 + n_2$)

- A. $O(1)$
- B. $O(n)$
- C. $O(n^2)$
- D. None of the above

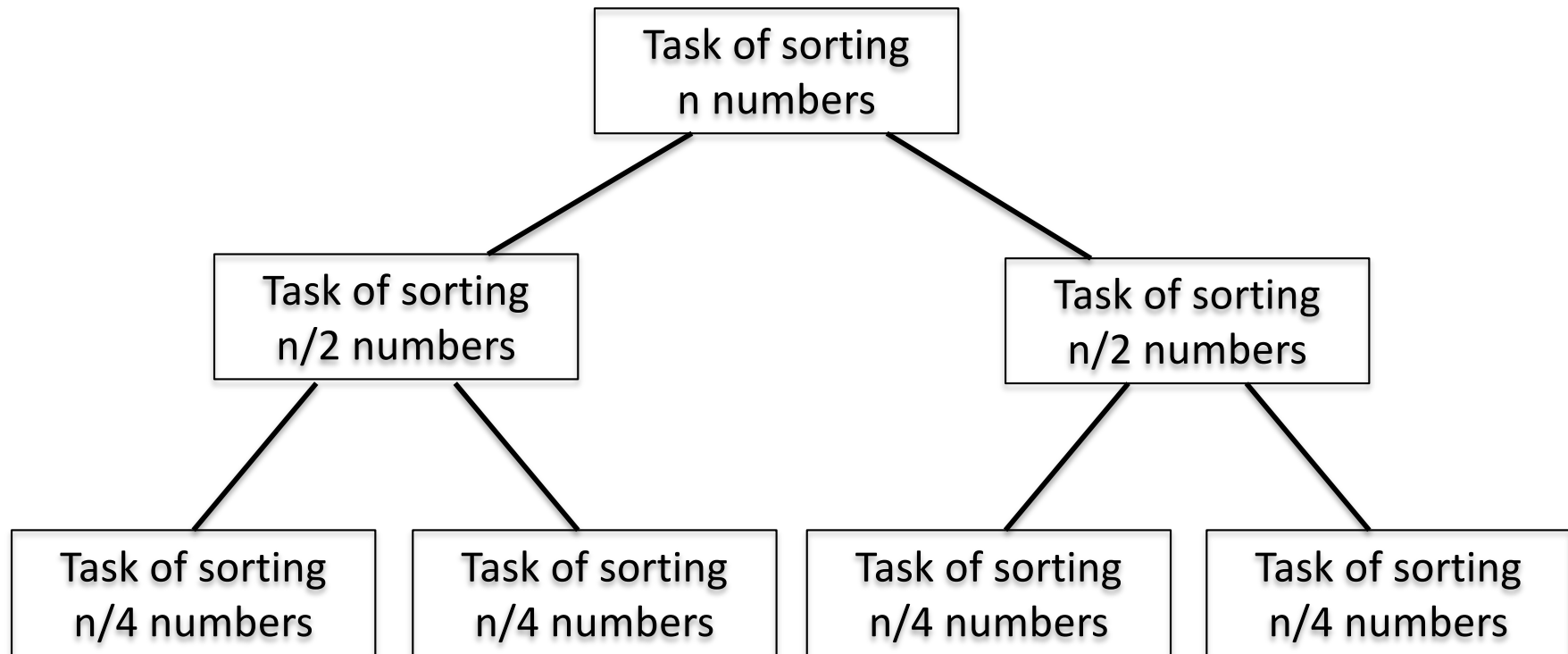
Running time of mergesort

Task of sorting
 n numbers

Running time of mergesort

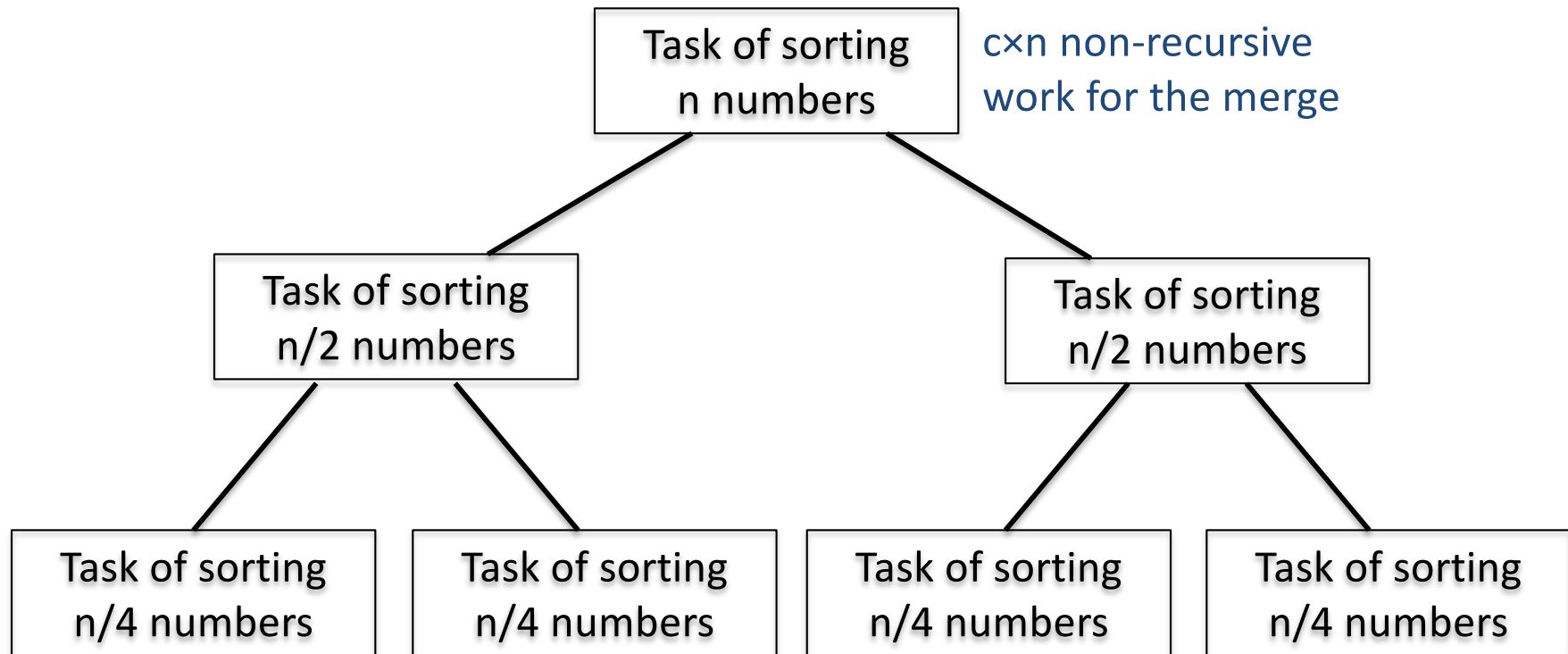


Running time of mergesort



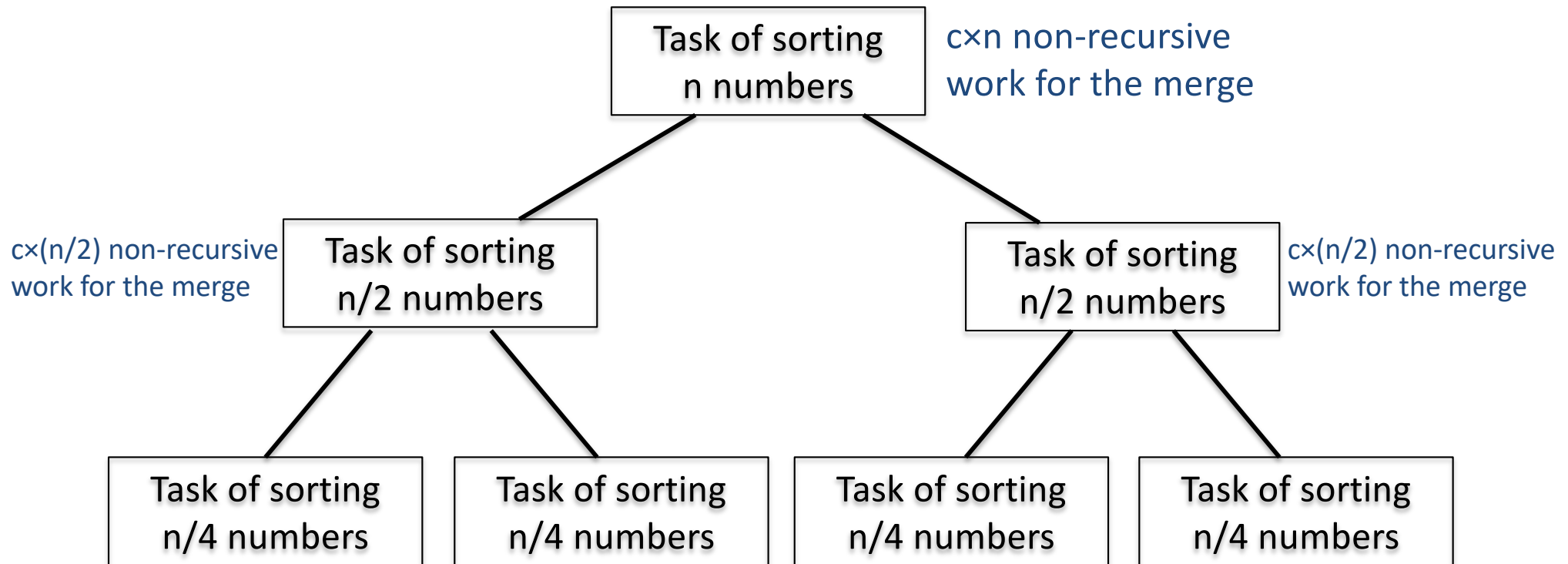
... and so on

Running time of mergesort



... and so on

Running time of mergesort



... and so on

Logarithms!

Logarithms

$\log_b x$ = “log base b of x”

= power of b that gives x

= number of times you can divide x by b
before getting 1

Logarithms

$\log_b x$ = “log base b of x”

= power of b that gives x

= number of times you can divide x by b
before getting 1

$2^0 = 1$ so $\log_2 1$ is 0

$2^1 = 2$ so $\log_2 2$ is 1

$2^2 = 4$ so $\log_2 4$ is 2 (and $\log_2 3$ is between 1 & 2)

What is $\log_2 32$?

- A. 3
- B. 4
- C. 5
- D. 6
- E. None of the above

What is $\log_2 32$?

A. 3

B. 4

C. 5

D. 6

E. None of the above